06회차 Interrupt 수집 개선 요청서



TC387QP · ISR · 공유 데이터 · Pull Request

01 / REQUEST

주기 수집 요청이 사라지는 원인을 찾아 주세요.

> 요청 범위: 10ms 주기 요청을 짧은 ISR로 기록하고, Main에서 최신 센서값을 한 번 읽어 보고하는 기능을 안정화해 주세요.

개발팀은 TC387QP 기반 수집 기능을 Polling 방식에서 Timer Interrupt 방식으로 변경하고 있습니다. 현재 시제품 코드에는 ISR 안의 느린 작업과 Main·ISR 사이의 공유 변수 처리가 섞여 있습니다. 요청이 겹칠 때의 누락, 처리 개수와 Timestamp의 불일치, Interrupt 차단 구간을 확인하고 개선해 주세요.

학습 목표

목표	이번 과제에서 남길 증거
Polling과 Interrupt의 처리 흐름을 비교하고 사용 조건을 정한다.	두 흐름도와 선택 이유. Main이 늦는 상황과 Interrupt가 차단되는 상황을 구분한다.
Timer ISR에서 짧게 처리할 일과 Main 흐름으로 넘길 일을 나눈다.	timer_isr.c와 collector.c의 개선 코드, I/O 실행 Context와 호출 횟수 Test.
공유 데이터의 Race condition을 찾고 Atomicity·Critical section과 지연의 관계를 검토한다.	읽기·초기화 사이의 Interrupt 재현, 이전 IRQ 상태 보존, 지연 분석.

납품물은 개선한 세 C Module, 직접 추가한 Test, 근거와 실행 Log를 포함한 Pull Request입니다. 전체 Firmware나 RTOS를 새로 구현하지 않습니다.

06회차 Interrupt 수집 개선 요청서



TC387QP · ISR · 공유 데이터 · Pull Request

02 / INPUTS & EVIDENCE

문서에서 확인할 사실과 제품 정책을 구분합니다.

구분	확인할 내용
Datasheet	TC387QP가 속한 TC38x 계열과 STM 주변장치의 존재를 확인합니다. 제품 이름만으로 Board의 Clock 값을 추정하지 않습니다.
User Manual	STM의 free-running Counter, Compare, Interrupt 요청 경로와 CPU의 Interrupt 제어 범위를 찾습니다.
공식 예제 / iLLD	STM_Interrupt_1_KIT_TC387_TFT에서 Compare 주기, ISR 선언, TOS와 Priority 설정을 추적합니다. IfxCpu_disableInterrupts / restoreInterrupts의 계약도 확인합니다.
이번 요청서	10ms, Mailbox 용량 4, Overflow와 병합 정책은 이번 제품의 요구 조건입니다. Datasheet가 정해 준 값으로 적지 않습니다.

대상은 단일 CPU0에서 실행되는 Main과 한 Timer ISR입니다. 중첩 ISR·다른 Core·DMA의 동시 Mailbox 접근은 없습니다. Main의 collector_step 호출 시 IRQ는 enabled 상태이며, Critical section 안에서 임의로 IRQ를 enable하지 않습니다.

Host C11 프로그램은 한 Thread의 결정적 Hook으로 Interrupt 진입 시점을 재현합니다. IRQ가 disabled이면 한 개의 pending IRQ만 보류합니다. 여러 차단 중 요청이 모두 복구되는 Queue가 아닙니다.

> Host Test는 실제 TC387QP Binary, MMIO 접근, Board 실행, 지연 실측을 검증하지 않습니다.
> 10ms 전에 다음 Compare를 설정할 수 있다는 Hardware 조건은 별도로 확인해야 합니다.

고정 공식 예제: STM_Interrupt_1_KIT_TC387_TFT

세부 문서 위치와 링크는 함께 제공한 REFERENCES.md에 있습니다. 문서명·절·함수명과 해석을 근거로 남겨 주세요.

06회차 Interrupt 수집 개선 요청서



TC387QP · ISR · 공유 데이터 · Pull Request

03 / MAILBOX CONTRACT

수집 요청의 개수와 마지막 시각을 함께 넘깁니다.

```
EventMailbox: volatile uint32_t pending, dropped, last_tick
EventBatch:   uint32_t pending, dropped, last_tick
Capacity:     4 requests
```

API / 조건	요구 동작
event_post_isr(tick) pending 〈 4	pending을 1 증가시키고 last_tick에 수락한 요청의 tick을 기록합니다.
event_post_isr(tick) pending == 4	pending과 last_tick을 보존하고 dropped를 증가시킵니다. dropped는 UINT32_MAX에서 포화시켜 Wraparound를 막습니다.
event_take(NULL)	false를 반환하며 공유 상태·출력·IRQ 상태에 영향을 주지 않습니다.
event_take(out) 비어 있음	false를 반환하고 out의 기존 값을 보존합니다. 이전 IRQ 상태도 보존합니다.
event_take(out) 요청 있음	세 필드를 일관된 Snapshot으로 가져와 공유 필드를 모두 0으로 만듭니다. 읽기와 초기화는 하나의 Critical section에서 완료해야 합니다. out에 Snapshot을 쓰고 true를 반환합니다.

event_take의 out은 Mailbox와 별도 저장공간입니다. 제공된 platform_interleave() 호출은 공유 필드 읽기 직후·초기화 직전에 정확히 한 번 유지합니다. Test 관찰 지점이므로 삭제하거나 위치를 옮겨 실패를 숨기지 않습니다.

> restore 시 보류된 ISR이 바로 실행될 수 있습니다. 이번 Snapshot을 반환하는 동안 들어온 다음 요청을 지우지 말아 주세요. event_reset은 Timer 정지 또는 IRQ 차단 상태에서만 쓰는 제공 초기화 함수입니다.

06회차 Interrupt 수집 개선 요청서



TC387QP · ISR · 공유 데이터 · Pull Request

04 / ISR & MAIN CONTRACT

ISR은 요청을 기록하고 Main은 실제 값을 읽습니다.

API	요구 동작
timer_isr()	platform_timer_service()를 첫 동작으로 정확히 한 번 호출합니다. 이어서 platform_timer_now()로 얻은 시각을 event_post_isr에 전달합니다. Sensor read·Publish·Delay는 호출하지 않습니다.
collector_step(NULL)	false. Mailbox를 소모하거나 I/O를 수행하지 않습니다.
collector_step(out) 빈 Batch	event_take를 한 번 호출합니다. false이면 out을 보존하고 Sensor read·Publish 없이 종료합니다.
collector_step(out) 유효 Batch	Critical section 종료 후 센서를 한 번 읽고 Report를 한 번 Publish합니다. 둘 다 Main·IRQ enabled 상태에서 실행해야 합니다. out에 동일 Report를 쓰고 true를 반환합니다.

```
SampleReport:
  request_tick = batch.last_tick
  requests     = batch.pending
  coalesced    = batch.pending - 1
  dropped      = batch.dropped
  value        = current sensor value (0..4095)
```

여러 요청을 받았어도 센서는 현재 값을 한 번만 읽습니다. coalesced는 이 정책에 따라 합친 수락 요청 수이며 dropped는 용량 초과로 거부한 요청 수입니다. request_tick은 실제 측정 시각이 아니라 마지막으로 수락한 ISR 요청의 시각입니다.

> platform_timer_service는 Host에서 호출 순서를 검증하고 다음 Compare 서비스 경계를 나타내는 추상화입니다. 특정 STM Flag를 지우지 않으면 항상 재진입한다는 의미가 아닙니다.

06회차 Interrupt 수집 개선 요청서



TC387QP · ISR · 공유 데이터 · Pull Request

05 / ACCEPTANCE CASES

다음 결과를 재현하고, 빠진 경계를 추가해 주세요.

입력 순서	기대 결과
빈 Mailbox에서 collector_step	false, out 보존, Sensor 0회, Publish 0회.
요청 tick 10, 20, 30 이후 Main 처리	requests=3, request_tick=30, coalesced=2, dropped=0. Sensor·Publish 각 1회.
요청 tick 10..60, 10 간격 이후 Main 처리	requests=4, request_tick=40, coalesced=3, dropped=2. Sensor·Publish 각 1회.
tick 10을 읽은 직후 Hook에서 tick 20 요청	첫 Snapshot은 tick 10의 요청 1개. Critical section 종료 후 tick 20이 다음 Snapshot에 남아야 함.
IRQ disabled 상태에서 event_take	처리 후에도 IRQ disabled. 내부에서 무조건 enable하지 않음.
Sensor 또는 Publish 중 새 요청	현재 Report를 오염시키지 않고 다음 collector_step에서 처리.
pending=4, dropped=UINT32_MAX 추가 ISR 요청	pending·last_tick 보존. dropped가 0으로 돌아가지 않음.

읽기·초기화의 원자성을 단일 변수의 volatile 선언만으로 설명하지 마세요. 어떤 두 동작 사이에 ISR이 들어오면 어떤 값이 사라지는지 실제 순서를 제시해야 합니다.

위 tick은 Host에 주입한 가상 단위입니다. 기본 시나리오는 ms로 해석합니다. 10..60의 여섯 번은 ISR이 실제 실행된 요청이며 Hardware가 놓친 주기를 뜻하지 않습니다.

06회차 Interrupt 수집 개선 요청서



TC387QP · ISR · 공유 데이터 · Pull Request

06 / INVESTIGATION & VERIFICATION

실패 → 근거 → 변경 → Test를 연결합니다.

01 원본의 make test 결과를 먼저 기록합니다. 실패한 관찰값을 요구 조건과 연결하고, 같은 근본 원인에서 나온 중복 증상은 구분합니다.

02 Polling과 Interrupt 흐름도를 각각 그립니다. Main 지연, Interrupt 차단, ISR 실행 시간이라는 세 상황에서 무엇을 놓칠 수 있는지 설명합니다.

03 event_mailbox.c, timer_isr.c, collector.c를 개선합니다. API·자료형·Host·제공 Test·Hook 계약은 유지하고 private Helper는 추가할 수 있습니다.

04 tests/test_student.c에 서로 다른 경계 Test를 최소 2개 추가합니다. 정상 입력 반복만으로 채우지 말고 예상 결과를 코드 수정 전에 적습니다.

05 make test와 make demo를 실행하고 실제 결과를 저장합니다. CONCURRENCY.md에 경쟁 순서·IRQ 복구·지연 분석과 공식 근거를 작성합니다.

지연 분석에 사용할 가정

요청 주기는 10ms입니다. 비교용으로 Sensor read 2ms, Publish 15ms를 가정합니다. 이를 ISR 또는 IRQ 차단 구간에서 처리할 때와 Main에서 IRQ를 허용한 채 처리할 때를 비교해 주세요. 숫자는 분석용 가정이며 Test나 MCU 측정값이 아닙니다.

개선한 짧은 Critical section의 실행 시간을 B라고 두고 이 구간이 추가하는 최대 차단 시간을 설명합니다. B가 0인지, 총 Interrupt 지연까지 B로 단정할 수 있는지, 어떤 Hardware 측정이 추가로 필요한지도 적습니다.

> 모든 근본 원인을 설명해야 합니다. 단순히 Test가 통과했다는 결론이나 volatile을 붙였다는 이유만으로 원자성과 지연 요구를 충족했다고 판단하지 않습니다.

06회차 Interrupt 수집 개선 요청서



TC387QP · ISR · 공유 데이터 · Pull Request

07 / DELIVERY

코드·근거·실행 결과를 하나의 PR로 제출합니다.

위치	제출 내용
src/ + tests/	개선한 세 C Module과 고정 Test·추가 Test. 제공 include/·host/·Makefile도 함께 복사합니다.
README.md	증상, 실패 입력과 실제 결과, 모든 근본 원인, 수정 이유, 검증 결과.
CONCURRENCY.md	Polling / Interrupt 흐름도, ISR / Main 업무 분리, Race timeline, IRQ 복구, 17ms 가정과 B의 지연 분석, 근거 위치.
BUILD_ENV.md + logs/	OS·Compiler·Make·옵션·기준 Source와 실제 명령/Exit code. 수정 전·후 Test Log 및 Demo CSV.

```
submissions/week06/YOUR_GITHUB_ID/
  src/              tests/
  include/          host/
  Makefile          .gitignore
  README.md         CONCURRENCY.md
  BUILD_ENV.md      logs/
```

원본 weeks/week06/starter는 그대로 보존합니다. 자신의 Fork에서 Branch를 만들고 자기 제출 폴더만 변경하여 zinee-u/embedded-just-for-fun의 main으로 PR을 보냅니다. 자세한 명령은 SUBMISSION.md, 본문 양식은 PR_BODY.md를 사용합니다.

공개된 풀이 접근법에서 학습 목표와 공식 근거, 실패 원인을 확인할 수 있습니다. 이를 참고해 자신의 코드·추가 Test·실행 결과를 작성하고 자기 제출 폴더로 PR을 보내 주세요. 제출 마감일은 별도 안내합니다.

06회차 공개 풀이 접근법

제출 대상 저장소